# 操作系统原理实验报告

| 实验名称 | 授课老师 | 学生姓名 | 学生学号 |
| --- | --- | --- | --- |
| 实模式和保护模式下的OS启动 | 张青 | 王健阳 | 21307261 |

## 一、实验要求

初步了解实模式和保护模式下的OS启动

了解计算机开机启动的大致流程

实现实模式下的中断，并用中断实现光标位置的获取及移动

掌握x86汇编基础

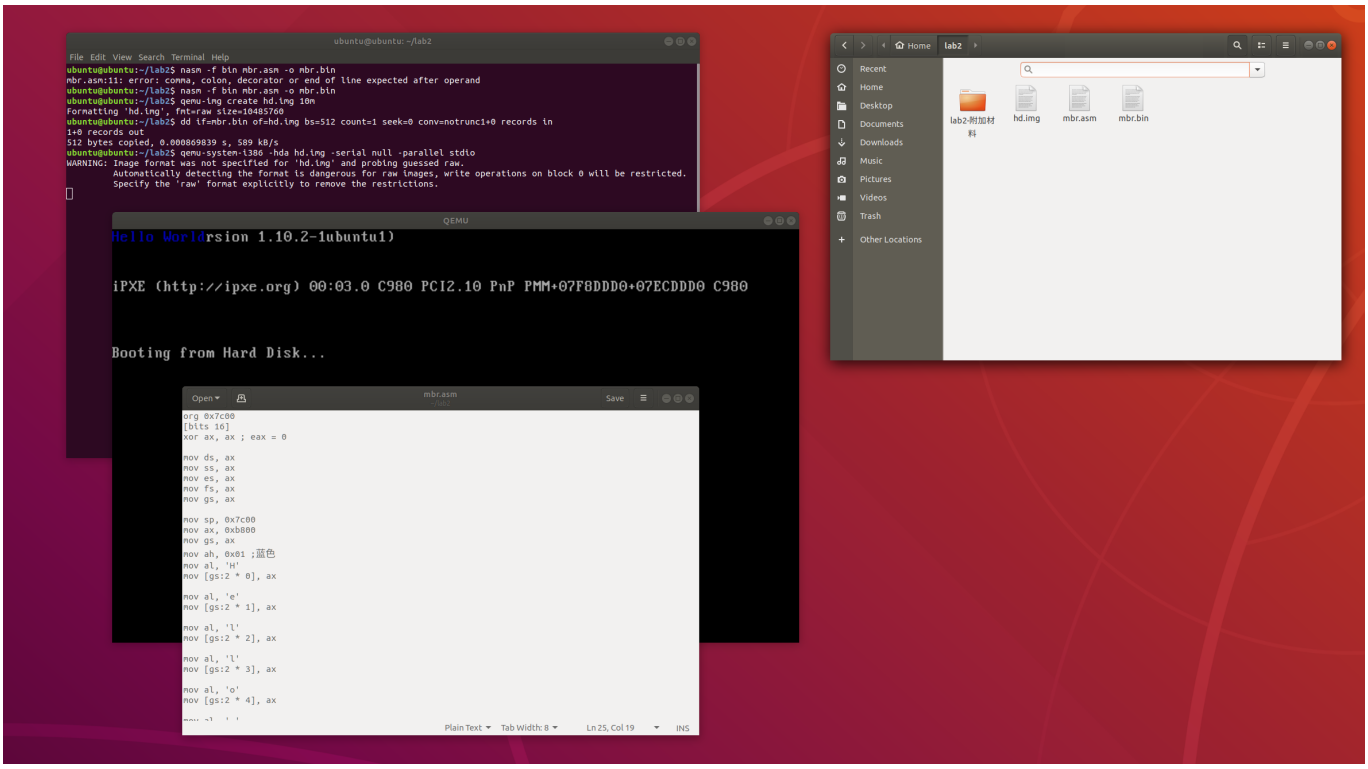
完成四个assignment

## 二、实验过程

**assignment 1 MBR**

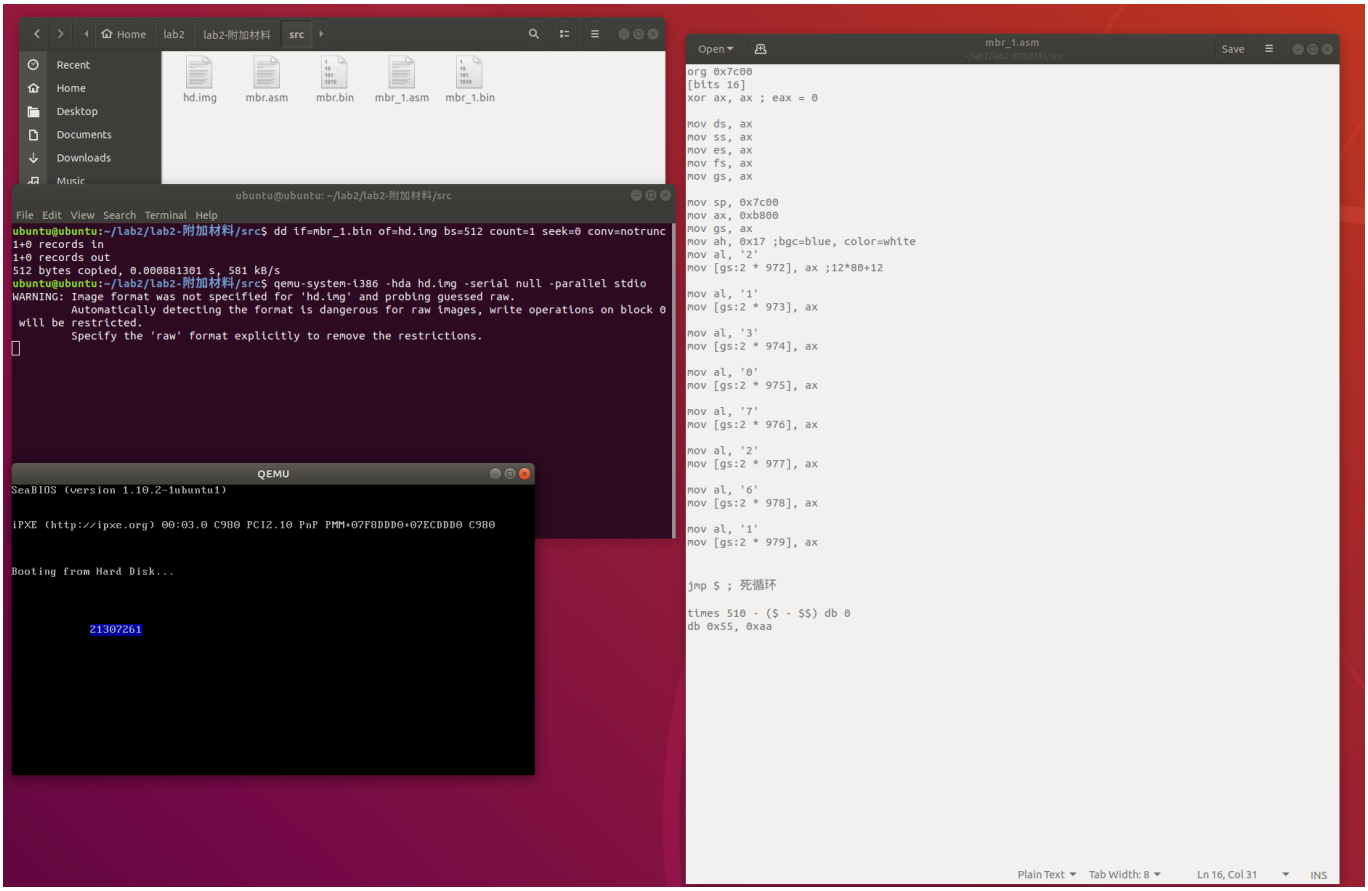
实验过程：按照pdf上的步骤连接上qemu，并在显示屏上输出相关内容

> 1.1 输出helloworld




按照example1的流程，依次运行指令得以复现（hello world为深蓝色不太好看出）

> 1.2 在1.1基础上在（12,12）处输出自己的学号




通过改变ah的值来实现背景色和颜色的改变。高四位决定bgc，第四位决定color

（按照pdf上的对应规则）

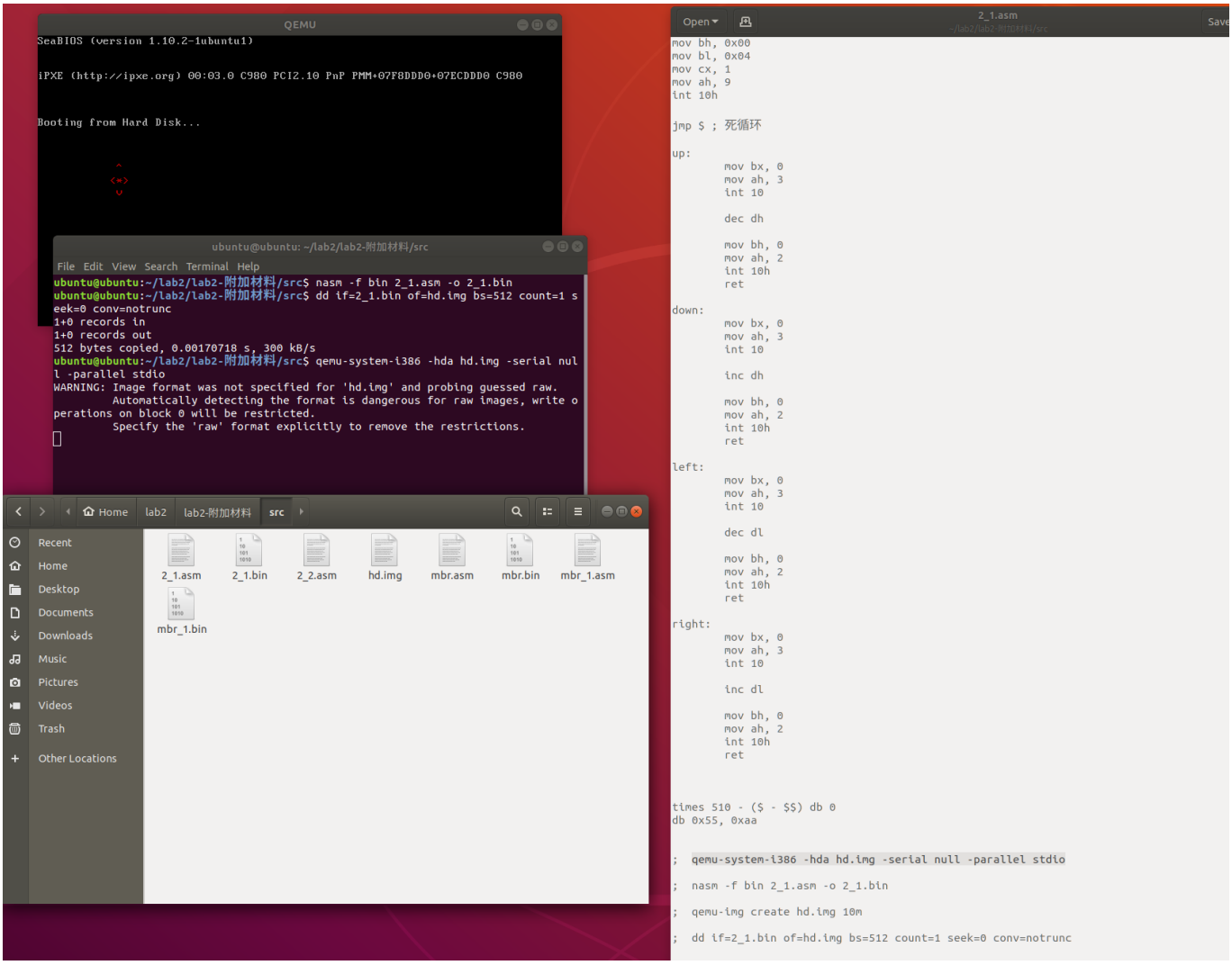
然后起始地址改为（12*80 + 12）

**assignment 2 实模式中断**

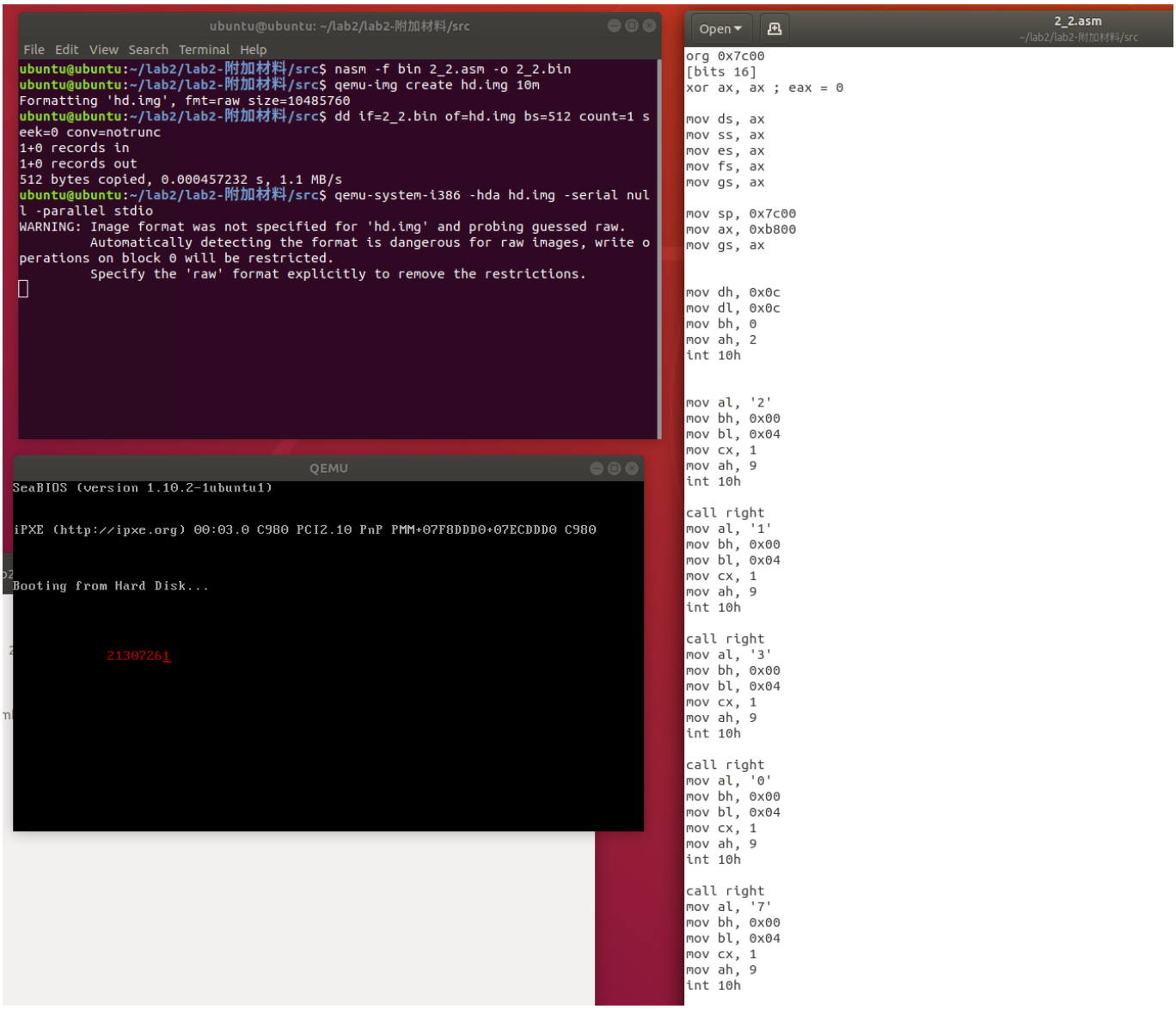
第二部分实验流程：

1. 编写汇编文件
2. 运行qemu查看结果
3. debug

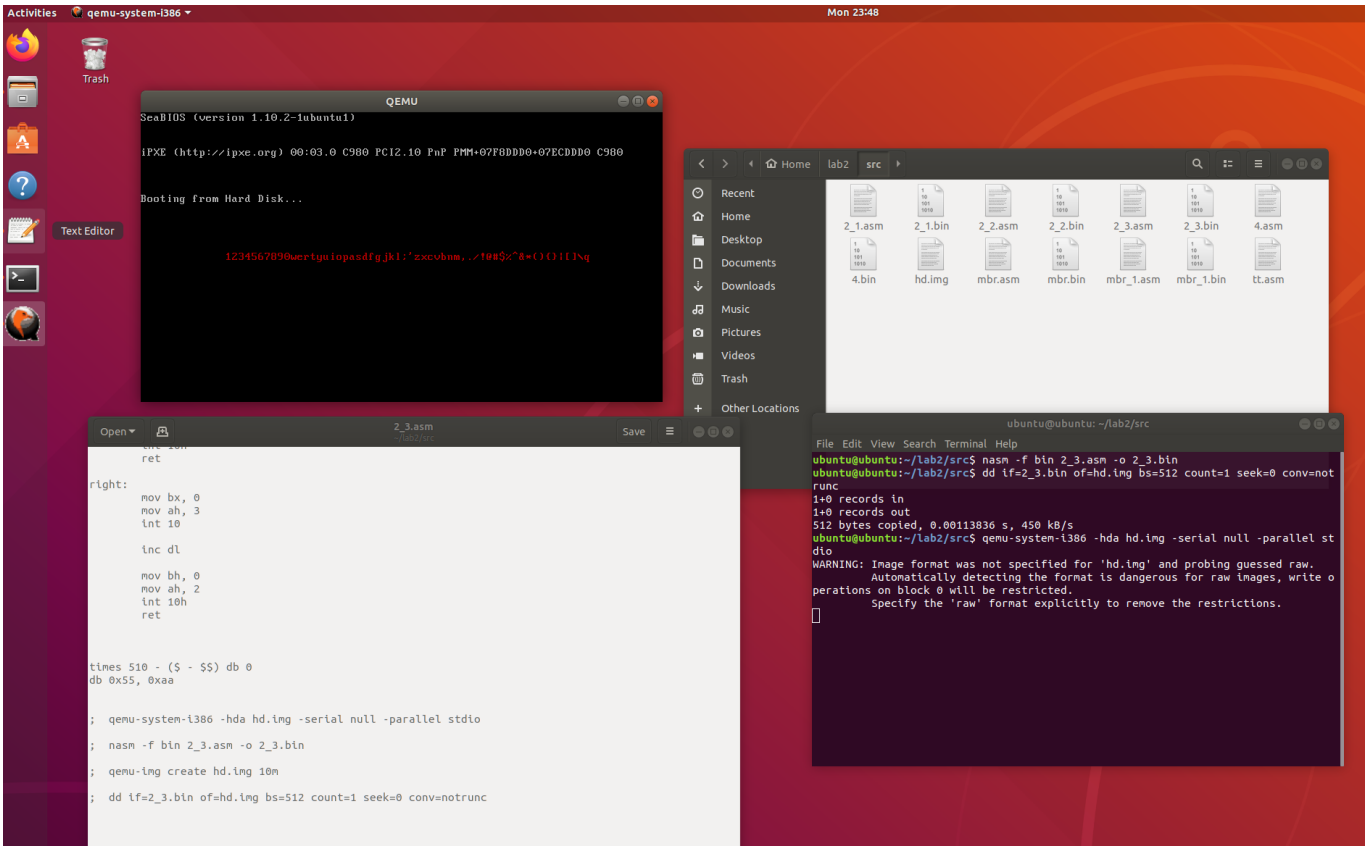
## 2.1 利用中断实现光标的位置获取以及移动




## 2.2 基于中断输出自己的学号

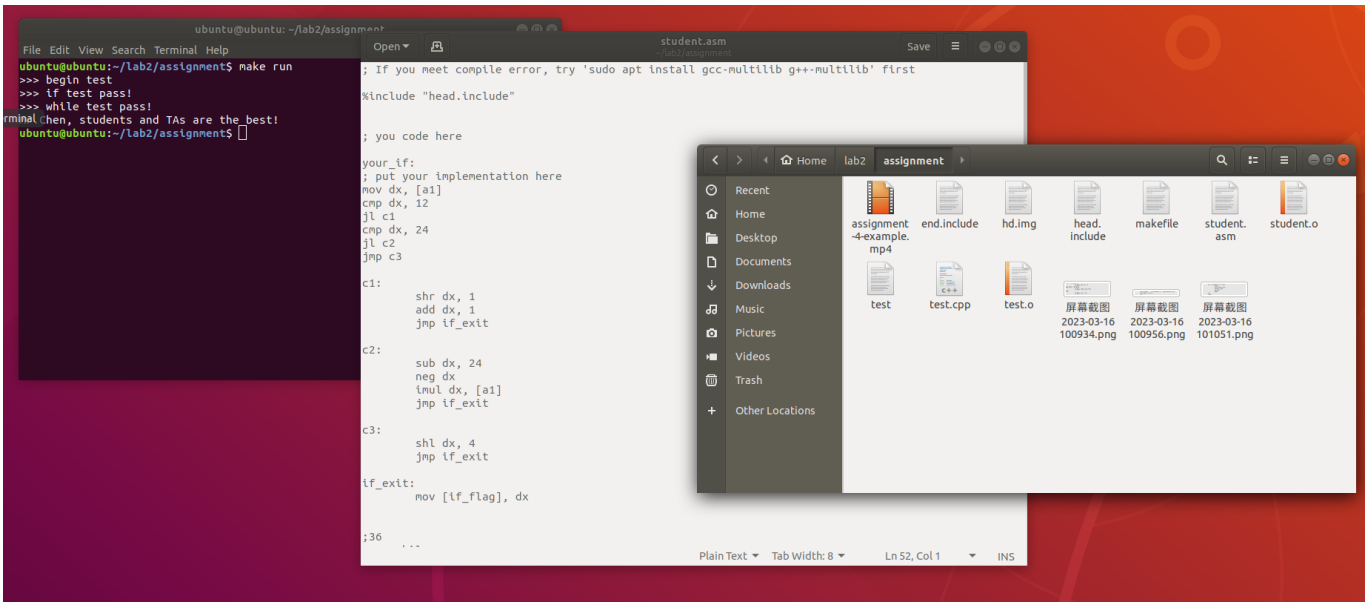



## 2.3 利用键盘中断实现键盘输入并回显




## assignment 3 汇编

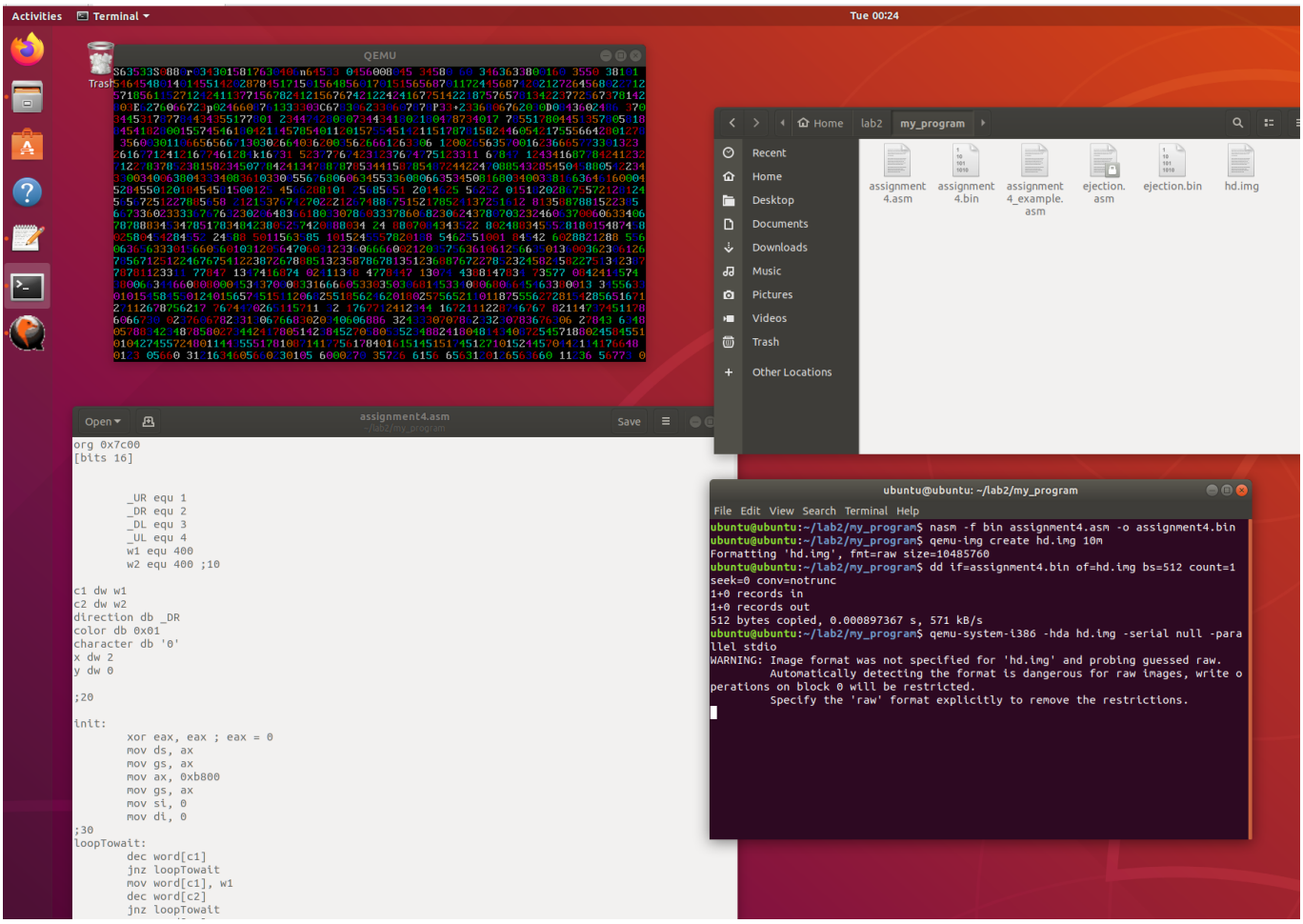
实验过程：完成student.asm并debug




## assignment 4 汇编小程序

实验过程：编写asm并运行debug

（因为截图时间过晚，此时字符已经充满了整个屏幕）




# 三、关键代码

**assignment 1**

（因为1.1代码完全由pdf提供，直接复制粘贴即可。故此处只展示1.2代码）

```
org 0x7c00
[bits 16]
xor ax, ax ; eax = 0

mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax

mov sp, 0x7c00
mov ax, 0xb800
mov gs, ax
mov ah, 0x17 ;bgc=blue, color=white
mov al, '2'
mov [gs:2 * 972], ax ;12*80+12

mov al, '1'
mov [gs:2 * 973], ax
```

```
mov al, '3'
mov [gs:2 * 974], ax

mov al, '0'
mov [gs:2 * 975], ax

mov al, '7'
mov [gs:2 * 976], ax

mov al, '2'
mov [gs:2 * 977], ax

mov al, '6'
mov [gs:2 * 978], ax

mov al, '1'
mov [gs:2 * 979], ax


jmp $ ; 死循环

times 510 - ($ - $$) db 0
db 0x55, 0xaa
```

**assignment 2**

> 2.1

```
org 0x7c00
[bits 16]
;寄存器初始化
xor ax, ax

mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax

mov sp, 0x7c00
mov ax, 0xb800
mov gs, ax
;设置光标的初始位置为 (12,12)
mov dh, 0x0c
mov dl, 0x0c
mov bh, 0
mov ah, 2
int 10h

mov al, '*'
mov bh, 0x00 ;设置为黑底红字
```

```
mov bl, 0x04
mov cx, 1 ;设置为输出一次
mov ah, 9
int 10h

call right ;光标右移
mov al, '>'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h

call left ;光标左移
call left
mov al, '<'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h

call right
call up ;光标上移
mov al, '^'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h

call down
call down ;光标下移
mov al, 'v'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h

jmp $ ; 死循环

up:
    mov bx, 0
    mov ah, 3
    int 10 ;获取光标位置
    dec dh ;横坐标减一
    mov bh, 0 ;重新设置光标位置
    mov ah, 2
    int 10h
    ret
down:
    mov bx, 0
    mov ah, 3
```

```
    int 10 ;获取光标位置
    inc dh ;横坐标加一
    mov bh, 0 ;重新设置光标位置
    mov ah, 2
    int 10h
    ret
left:
    mov bx, 0
    mov ah, 3
    int 10 ;获取光标位置
    dec dl ;纵坐标减一
    mov bh, 0 ;重新设置光标位置
    mov ah, 2
    int 10h
    ret
right:
    mov bx, 0
    mov ah, 3
    int 10 ;获取光标位置
    inc dl ;纵坐标加一
    mov bh, 0 ;重新设置光标位置
    mov ah, 2
    int 10h
    ret

times 510 - ($ - $$) db 0
db 0x55, 0xaa
```

> 2.2

```
org 0x7c00
[bits 16]
;初始化各寄存器的值
xor ax, ax

mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax

mov sp, 0x7c00
mov ax, 0xb800
mov gs, ax

;设置光标的初始位置为 (12,12)
mov dh, 0x0c
mov dl, 0x0c
mov bh, 0
mov ah, 2
int 10h
```

```
;在当前光标位置输出‘2’
mov al, '2'
mov bh, 0x00
mov bl, 0x04;设置颜色为黑底红色，以下同理
mov cx, 1;输出一次，以下同理
mov ah, 9
int 10h
;光标右移
;在当前光标位置输出‘1’
call right
mov al, '1'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h
;光标右移
;在当前光标位置输出‘3’
call right
mov al, '3'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h
;光标右移
;在当前光标位置输出‘0’
call right
mov al, '0'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h
;光标右移
;在当前光标位置输出‘7’
call right
mov al, '7'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h
;光标右移
;在当前光标位置输出‘2’
call right
mov al, '2'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h
;光标右移
;在当前光标位置输出‘6’
```

```
call right
mov al, '6'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h
;光标右移
;在当前光标位置输出‘1’
call right
mov al, '1'
mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h

jmp $ ; 死循环

right:
    mov bx, 0
    mov ah, 3
    int 10 ;获取光标位置
    inc dl ;令光标的横坐标加一
    mov bh, 0 ;重新设置光标位置
    mov ah, 2
    int 10h
    ret

times 510 - ($ - $$) db 0
db 0x55, 0xaa
```

> 2.3

```
org 0x7c00
[bits 16]
xor ax, ax ; eax = 0

mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax

mov sp, 0x7c00
mov ax, 0xb800
mov gs, ax
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
mov dh, 0x0c
mov dl, 0x0c
mov bh, 0
mov ah, 2
```

```
int 10h

start:
;假如此处添加一个判断标志，可实现输入的换行
;或者自定义输入位置（让光标随机上下左右移动）
call right
mov ah, 0
int 16h ;调用中断，从键盘输入一个字符并显示

mov bh, 0x00
mov bl, 0x04
mov cx, 1
mov ah, 9
int 10h

cmp al, 'q' ;设置为输入q终止程序，可更改为其他字符
jne start

jmp $ ; 死循环

right:
    mov bx, 0
    mov ah, 3
    int 10
    inc dl
    mov bh, 0
    mov ah, 2
    int 10h
    ret

times 510 - ($ - $$) db 0
db 0x55, 0xaa
```

**assignment 3**

```
; If you meet compile error, try 'sudo apt install gcc-multilib g++-multilib'
first
%include "head.include"

; you code here

your_if:
; put your implementation here
mov edx, [a1] ;edx = a1
cmp edx, 12
jl c1 ;if a1 < 12 then
cmp edx, 24
jl c2 ;else if a1 < 24 then
jmp c3 ;else

c1:
```

```
    shr edx, 1
    add edx, 1
    jmp if_exit ;edx = a1 / 2 + 1

c2:
    sub edx, 24
    neg edx
    imul edx, [a1]
    jmp if_exit ;edx = (24 - a1) * a1

c3:
    shl edx, 4
    jmp if_exit ;edx = a1 << 4

if_exit:
    mov [if_flag], edx ;if_flag = edx


;因为以下两个数组都是指针变量，占四个字节，用到32位寄存器储存其值
your_while:
mov eax, [a2] ;eax = a2
; put your implementation here
    cmp eax, 12
    jl while_exit ;while a2 >= 12

    call my_random
    mov ebx, [a2]
    mov ecx, [while_flag]
    mov byte[ecx + ebx - 12], al ;while_flag[a2 - 12] = eax
                                 ;char类型只占一个字节，故只用al
    dec dword[a2] ;a2 --
    jmp your_while

while_exit:


%include "end.include"
your_function:
; put your implementation here
    pushad
    xor eax, eax ;eax = 0
loop:
    mov ecx, [your_string] ;ecx = &your_string[0]
    cmp byte[ecx + eax], 0
    je function_exit ;string[i] != '\0' -> continue to loop
    pushad
    mov ebx, dword[ecx + eax] ;将string[i]作为函数参数进栈
    push ebx
    call print_a_char
    pop ebx
    popad
    add eax, 1 ;eax += 1
    jmp loop
```

```
function_exit:
    popad
    ret
```

**assignment 4**

```
org 0x7c00
[bits 16]


    _UR equ 1
    _DR equ 2
    _DL equ 3
    _UL equ 4
    w1 equ 400
    w2 equ 400
;定义各变量的初始值
c1 dw w1
c2 dw w2
direction db _DR
color db 0x01
character db '0'
x dw 2
y dw 0
;初始化寄存器
init:
    xor eax, eax ; eax = 0
    mov ds, ax
    mov gs, ax
    mov ax, 0xb800
    mov gs, ax
    mov si, 0
    mov di, 0
;循环主函数，其在等待一段时间之后会继续运行
loopTowait:
    dec word[c1]
    jnz loopTowait
    mov word[c1], w1
    dec word[c2]
    jnz loopTowait
    mov word[c1], w1
    mov word[c2], w2
    ;循环结束，开始移动并输出
    ;根据al的值判断移动方向，并调用对应的函数
    mov al, 1
    cmp byte[direction], al
    jz UpRight

    mov al, 2
    cmp byte[direction], al
    jz DownRight
```

```
    mov al, 3
    cmp byte[direction], al
    jz DownLeft

    mov al, 4
    cmp byte[direction], al
    jz UpLeft
    jmp $


;考虑向右上移动。以及碰壁情况
UpRight:
    dec word[x]
    inc word[y]
    ;考虑碰到右壁的情形，y要重新赋值78
    mov bx, word[y]
    mov ax, 80
    sub ax, bx
    cmp ax, 0
    jz URToUpLeft
    ; 考虑碰到上壁的情形，x要重新赋值1
    mov bx, word[x]
    mov ax, 0
    sub ax, bx
    cmp ax, 0
    jg URToDownRight
    jmp show
URToUpLeft:
    mov word[y], 78
    mov byte[direction], _UL ;更改方向
    jmp show
URToDownRight:
    mov word[x], 1
    mov byte[direction], _DR ;更改方向
    jmp show



DownRight:
    inc word[x]
    inc word[y]
    ;考虑碰到右壁的情形，y要重新赋值78
    mov bx, word[y]
    mov ax, 80
    sub ax, bx
    cmp ax, 0
    jz DRToDownLeft
    ;考虑碰到下壁的情形，x要重新赋值23
    mov bx, word[x]
    mov ax, 25
    sub ax, bx
    cmp ax, 0
    jz DRToUpRight
```

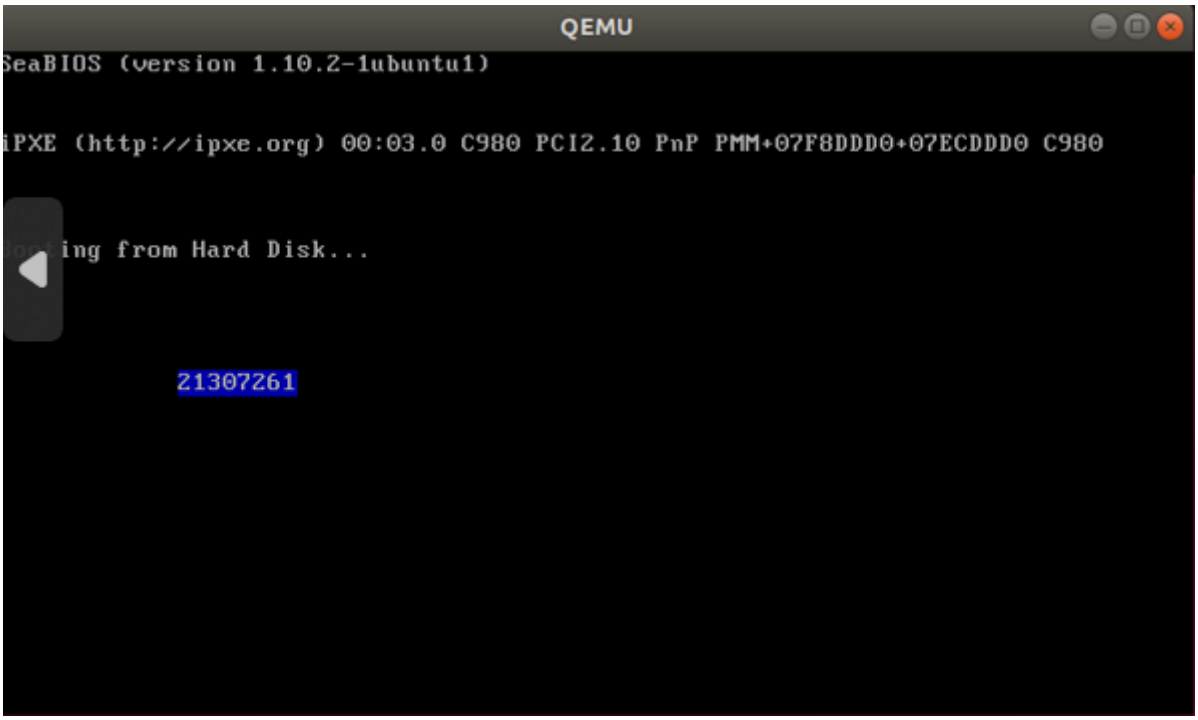
```
    jmp show
DRToUpRight:
    mov word[x], 23
    mov byte[direction], _UR ;更改方向
    jmp show
DRToDownLeft:
    mov word[y], 78
    mov byte[direction], _DL ;更改方向
    jmp show



DownLeft:
    inc word[x]
    dec word[y]
    ;考虑碰到下壁的情形，x要重新赋值23
    mov ax, 25
    mov bx, word[x]
    sub ax, bx
    cmp ax, 0
    jz DLToUpLeft
    ;考虑碰到左壁的情形，y要重新赋值1
    mov ax, 0
    mov bx, word[y]
    sub ax, bx
    cmp ax, 0
    jg DLToDownRight
    jmp show
DLToUpLeft:
    mov word[x], 23
    mov byte[direction], _UL ;更改方向
    jmp show
DLToDownRight:
    mov word[y], 1
    mov byte[direction], _DR ;更改方向
    jmp show



UpLeft:
    dec word[x]
    dec word[y]
    ;考虑碰到上壁的情形，x要重新赋值1
    mov ax, 0
    mov bx, word[x]
    sub ax, bx
    cmp ax, 0
    jg ULToDownLeft
    ;考虑碰到左壁的情形，y要重新赋值1
    mov ax, 0
    mov bx, word[y]
    sub ax, bx
    cmp ax, 0
    jg ULToUpRight
```

```
    jmp show
ULToDownLeft:
    mov word[x], 1
    mov byte[direction], _DL ;更改方向
    jmp show
ULToUpRight:
    mov word[y], 1
    mov byte[direction], _UR ;更改方向
    jmp show


show:
    ;计算出 MBR 地址。并放到bx中
    ;将需要打印的字符和颜色值存储进ax中
    xor ax, ax
    mov ax, word[x]
    mov bx, 80
    mul bx
    add ax, word[y]
    mov bx, 2
    mul bx
    mov bx, ax
    mov al, byte[character]
    mov ah, byte[color]
    mov [gs:bx], ax ;参考helloworld用例输出
    ;在上述点关于MBR显示屏中轴线的对称点出输出相同的字符，以实现双向输出
    xor ax, ax
    mov ax, word[x]
    mov bx, 80
    mul bx
    mov bx, word[y]
    neg bx
    add bx, 79
    add ax, bx
    mov bx, 2
    mul bx
    mov bx, ax
    mov al, byte[character]
    mov ah, byte[color]
    mov [gs:bx], ax

    ;更改颜色值，字符值，并分别令其始终在一定范围内
    inc byte[color]
    inc byte[character]
    cmp byte[color], 8
    jl color_next
    mov byte[color], 1
color_next:
    cmp byte[character], '9'
    jl character_next
    mov byte[character], '0'
character_next:
    jmp loopTowait
```

```
jmp $ ; 死循环

times 510 - ($ - $$) db 0
db 0x55, 0xaa
```

# 四、实验结果

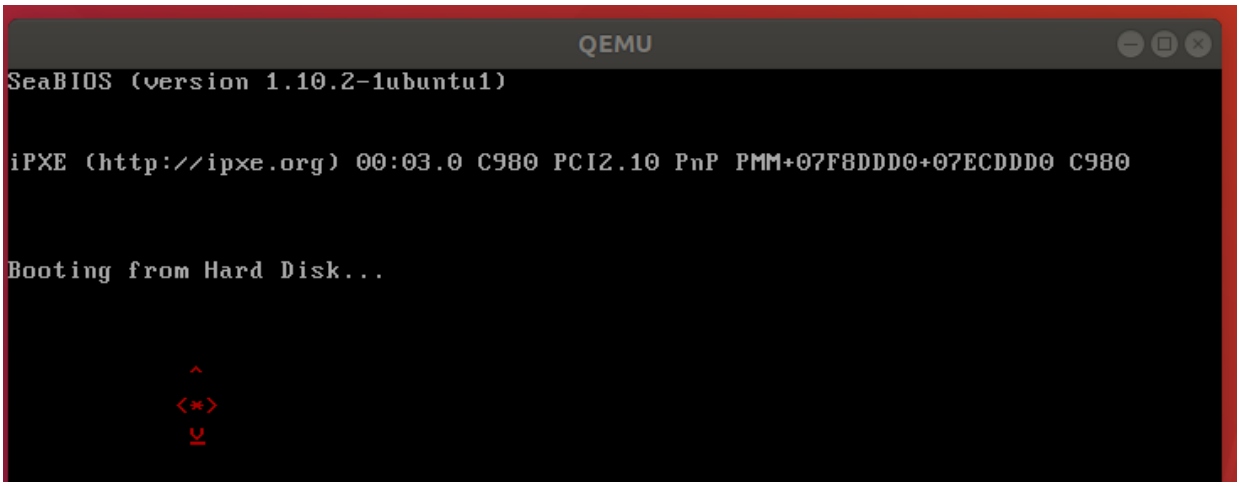
**assignment 1**




**assignment 2**

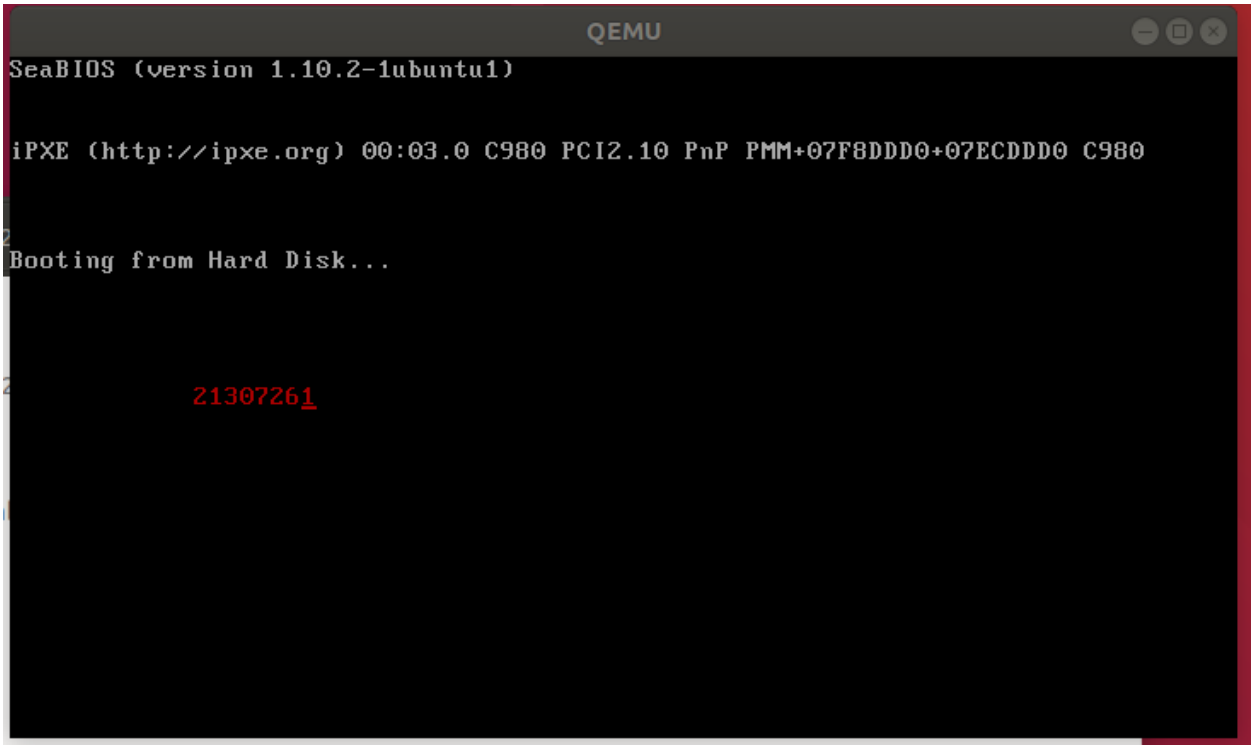
> 2.1

设置光标位置，并在相应位置输出字符，并实现光标的上下左右移动



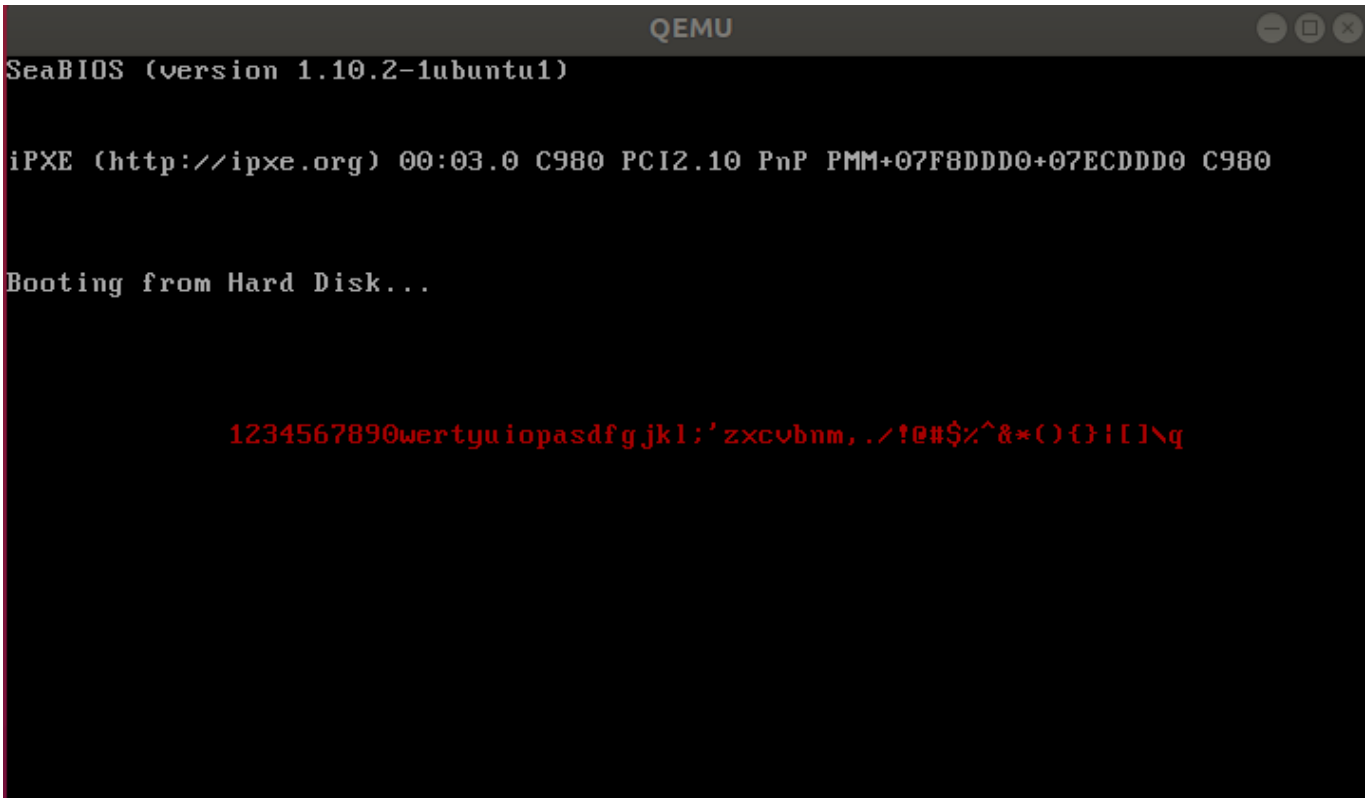

> 2.2

利用光标中断在（12,12）输出自己的学号



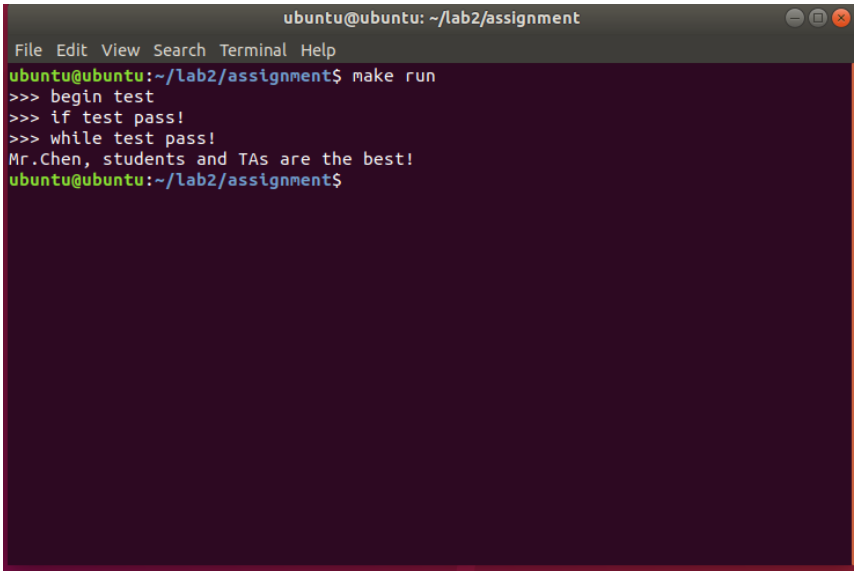

> 2.3

在（12,12）的位置输出了一系列字符，这里设置退出标志是q，故最后停止从键盘输入




**assignment 3**

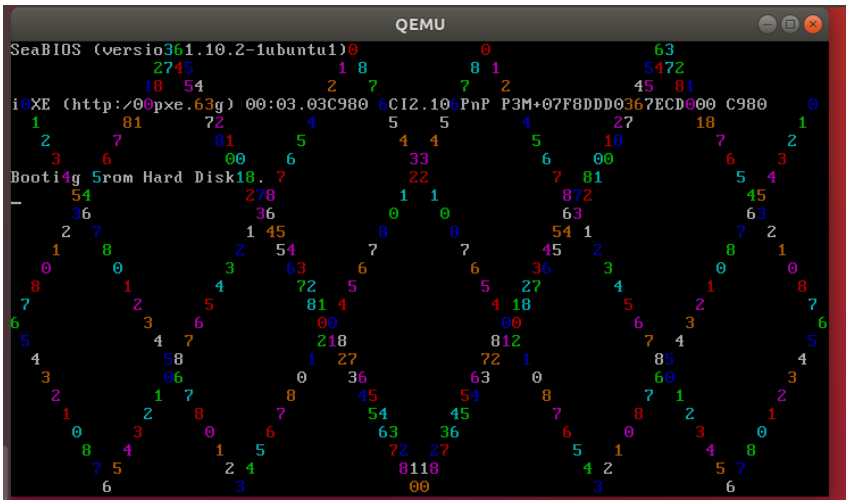
三个汇编测试均显示通过




**assignment 4**

汇编小程序的截图

实现了字符变色，依次输出0~9的字符，同时双向射出




# 五、总结

此次实验，让我对在实模式和保护模式下的OS启动有了一定的了解。任务一的直接输出比较简单，几乎一遍就过了。但是后面的三个任务都花了我不少时间。一开始我对中断没有操作概念，仅停留在理论层面的认识。但经过大量的资料查阅，我逐渐了解到这是个类似于函数的东西，也了解到了前两问应该调用的是10h中断。经过参数的正确排布之后，我也算是逐渐了解了如何利用中断去移动光标。

实验三让我对汇编的数据类型有了更深的认识，如果对其利用不当，很容易就会产生段错误。而任务四则重点在于程序的划分，如何划分可以让代码更简洁直观。对于碰壁的临界条件也需要仔细思索。还有就是函数名字高度重复，需要仔细debug，很容易打错）